高性能计算实验报告：判断性能瓶颈并提出优化方案

一、 实验目的与要求

· 下载 Linux C++17 stencil 与 GEMM 初始代码及依赖文件，在 Linux 中手动编译。 · 分别对照访存顺序（Row vs Column）和分块策略（Naive vs Blocked），给出计算量、内存流量模型与实验数据。 · 写出瓶颈判断和优化建议，包含正确性结果、平均时间、中位数、总体标准差、GFLOPS/s 和有效带宽等指标。

二、 实验环境与编译参数

· 操作系统：Linux 虚拟机 · 编译器：g++ (支持 C++17) · 编译参数（保持一致）：

```
g++ -O3 -march=native -std=c++17 stencil.cpp -o stencil
g++ -O3 -march=native -std=c++17 gemm.cpp -o gemm
```

· 参数说明：-O3 开启最高级别优化，-march=native 允许编译器自动使用当前机器支持的 SIMD 指令集（如 AVX2/AVX512），这对矩阵计算性能影响巨大。多次实验重复 10 次（REPEATS=10），包含 1 次预热（warmup=1）。

三、 理论模型推导（公式）

1. 计算量模型（FLOPs）

· Stencil (网格计算)：假设二维网格大小为 H \times W，迭代步数为 Steps，每个网格点每次更新约需 5 次浮点运算（1次乘加与周边读数）。 公式：Total\ FLOPs = H \times W \times Steps \times 5 代入数据：1000 \times 1000 \times 100 \times 5 = 500,000,000（0.5 GFLOPs） · GEMM (矩阵乘法)：大小为 N \times N 的方阵乘法，基础算法总乘加次数为 N^3，每次乘加算 2 次浮点运算。 公式：Total\ FLOPs = 2 \times N^3 代入数据：2 \times 1024^3 = 2,147,483,648（约 2.15 GFLOPs）

2. 内存流量模型（Bytes）

· Stencil 理想空间复用：完美空间复用下，数据只需从主存读取一次，写回一次。数据类型为 float（4字节）。 公式：Total\ Bytes = H \times W \times 4 \times 2 代入数据：1000 \times 1000 \times 4 \times 2 = 8,000,000 字节（8 MB） · GEMM 理论流量：需读入矩阵A、矩阵B，并写出结果矩阵C。 公式：Total\ Bytes = 3 \times N \times N \times 4 代入数据：3 \times 1024 \times 1024 \times 4 = 12,582,912 字节（约 12.58 MB）

3. 算术强度（Arithmetic Intensity, AI）

· 公式：AI = Total\ FLOPs \div Total\ Bytes · Stencil AI = 0.5G / 8MB ≈ 62.5 FLOPs/Byte · GEMM AI = 2.15G / 12.58MB ≈ 171 FLOPs/Byte

四、 实验数据汇总（实测结果）

注：以下数据由程序多轮运行自动统计，warmup=1 排除冷启动误差。程序自动校验结果（check=PASS），确保优化过程未导致结果错误。

实验名称	模式	平均时间(ms)	中位数(ms)	总体标准差	计算吞吐(GUpdates/s)	GFLOPS/s	有效带宽(GB/s)	正确性
Stencil	Row	18.5886	18.1292	1.07198	5.49392	26.91	0.43	PASS
Stencil	Column	124.298	123.524	3.59821	0.806324	4.02	0.064	PASS
GEMM	Naive	3923.75	3916.62	58.8232	不适用	0.5483	0.0032	PASS
GEMM	Blocked	127.486	128.079	2.10001	不适用	16.7669	0.0987	PASS

(注：Stencil的GFLOPS和带宽为结合理论公式推算，程序原生输出的是GUpdates/s；GEMM的GFLOPS为程序直接输出结果)

五、 瓶颈判断与数据分析

1. Stencil: Row vs Column（访存顺序影响）

· 数据分析：Column 模式的平均时间（124.30ms）是 Row 模式（18.59ms）的 6.7 倍左右，性能断崖式下跌。 · 原因分析（数据复用）：C/C++ 数组采用行优先存储。Row 模式访问连续内存，具有极佳的空间局部性，缓存命中率高。Column 模式每次跨步访问，缓存行利用率极低，频繁发生 Cache Miss（缓存未命中），导致 CPU 经常要停下计算去主存搬数据。 · 瓶颈判断：Stencil 的算术强度（AI=62.5）较低，在 Roofline 模型中更贴近内存带宽瓶颈。Column 模式严重受限于内存带宽利用率。

2. GEMM: Naive vs Blocked（分块策略影响）

· 数据分析：Naive 模式的平均时间高达 3923.75ms，而 Blocked（TILE=32）模式仅为 127.49ms，性能提升了约 30 倍，GFLOPS 从 0.55 飙升至 16.77。 · 原因分析（数据复用）：Naive 实现依赖三层大循环，数据在被用完后立刻被淘汰，时间局部性极差。Blocked 将大矩阵切分成 32x32 的小块，使得小块数据能够被完整加载到 L1/L2 缓存（案板）中，在缓存内反复复用，大幅减少了主存访问次数。 · 瓶颈判断：GEMM 的算术强度（AI=171）较高，属于计算密集型。Naive 实现受限于极差的内存访问模式（缓存颠簸），而 Blocked 优化解放了 CPU 算力。

六、 优化建议与模型差异

1. Stencil 优化建议：将 Column 遍历改为 Row 遍历（如涉及多维数组必须用行优先遍历）。若必须垂直扫描，建议采用时间分块 (Time Skewing)、或者引入多线程（OpenMP）最大化内存带宽利用率。
2. GEMM 优化建议：使用 Blocked 分块策略，并结合循环展开（Loop Unrolling）与 SIMD 显式指令（如 AVX）进一步压榨 CPU 算力，使得 GPU/CPU 计算单元达到满载。
3. 模型预测与真实测量的差距（提示3要求）：模型预测的 Roofline 上界通常高于真实硬件测量值。这是因为理论模型假设了绝对理想的内存访问（忽略 Cache Miss、TLB 未命中、内存延迟未隐藏等硬件物理开销）。此外，测试环境的系统抖动（标准差高达3.59ms或58.82ms）也印证了实际测量与理论模型的差异。因此，真实的优化需要不断逼近理论天花板，但无法完美达到。

七、 实验结论

通过本次实验，验证了访存顺序和分块策略对程序性能的巨大影响。结合算术强度与 Roofline 模型，成功判定 Stencil 为内存带宽受限，GEMM 为计算/缓存受限。优化决策需要对症下药：内存受限优化访存模式与分块复用，计算受限优化指令集与并行度。